

getarp.net — Defence Intelligence PoC: High-Level Design

Author: Security Architecture (15y) + 2 senior engineers **Status:** PoC / v0.4 **Target:** Single VM, 4 vCPU / 8 GB RAM / 100 GB disk, domain getarp.net

1. Objective

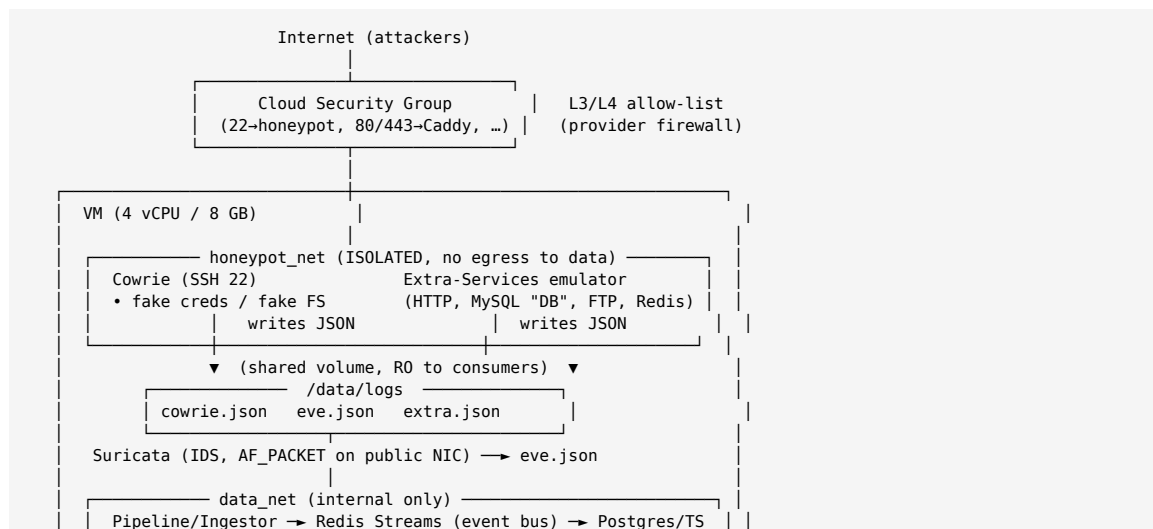
Stand up an internet-exposed deception sensor (Cowrie + multi-service emulator), capture real attacker traffic, run an IDS over it, enrich every observed IP with external threat intelligence from five providers (CrowdSec, AbuseIPDB, GreyNoise, VirusTotal, Abuse.ch Feodo Tracker) queried in parallel, correlate scans vs. attacks, profile attacker behaviour, and surface all of it through a public dashboard and an authenticated admin backend — all on one VM, built to be modular so components (intel providers, correlation engine, an AI module) can be swapped or added later.

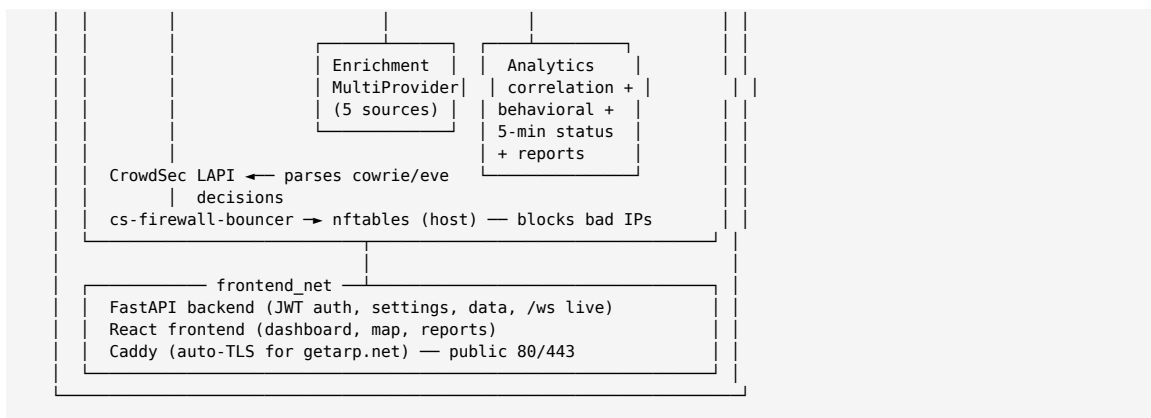
2. The one principle that drives the whole design

A honeypot is built to be compromised. Treat the sensor as hostile.

Everything else follows from this. The internet-facing deception layer is placed in an isolated network segment that **cannot reach** the data store, the API, or the admin plane. Telemetry leaves the honeypot the only safe way it can: as **append-only log files on a read-only-to-consumers volume**, pulled by a collector. There is no network path from a popped honeypot into your intelligence database.

3. Logical architecture





Network segmentation is enforced with three Docker networks. The honeypot network is internal: false only on the published attack ports; it has **no member** that also sits on data_net. The pipeline never connects *to* the honeypot — it only reads the shared log volume.

4. Component responsibilities

Layer	Component	Tech	Role
Deception	Cowrie	cowrie/cowrie	Medium-int fake FS, fal
Deception	Extra-Services	custom asyncio	HTTP/MySQ banners, lo
Detection	Suricata	jasonish/suricata	IDS on the Open + cu: eve.json
Intel/Enforcement	CrowdSec	crowdsecurity/crowdsec	Parses sens decisions, (community
Enforcement	cs-firewall-bouncer	bouncer	Applies Cro to host nfta
Bus/Cache	Redis	redis:7	Event strea live status
Storage	PostgreSQL + TimescaleDB	timescale/timescaledb	Events, ses enrichment reports, us
Ingest	Pipeline	Python	Tails sensor → Redis str
Intel	Enrichment — MultiProvider	Python	Queries all parallel; m reputation; provider's r
Analytics	Analytics	Python	Pluggable (scan/attac profiling, 5- reports

Layer	Component	Tech	Role
API	Backend	FastAPI	Auth, setting endpoints, feed
UI	Frontend	React + Vite	Dashboard correlation, reports, settings
Edge	Caddy	caddy:2	Reverse proxy, HTTPS for cloud

5. Canonical event schema (the contract that makes it modular)

Every sensor is normalized to one schema before anything downstream touches it. Swap a sensor or a provider, and nothing else changes as long as it emits/consumes this:

```
{
  "event_id": "uuid",
  "ts": "RFC3339",
  "sensor": "cowrie|suricata|extra",
  "service": "ssh|http|mysql|ftp|redis|...",
  "event_type": "connect|login_attempt|login_success|command|alert|scan|file",
  "src_ip": "x.x.x.x",
  "src_port": 0,
  "dst_port": 22,
  "username": null,
  "password": null,
  "command": null,
  "signature": null,
  "severity": 0,
  "session": "cowrie-session-id",
  "raw": { }
}
```

`event_type` + `service` are the only fields most downstream modules need. The `raw` blob preserves the original record for forensics and for future modules (e.g. an AI module that wants the full command transcript).

6. Data flow

1. Attacker hits Cowrie / Extra-Services / triggers a Suricata rule.
2. Sensors append JSON to `/data/logs/*.json`.
3. **Pipeline** tails those files, normalizes to the canonical schema, XADDS to the Redis stream events, and bulk-inserts into the events hypertable. New IPs are pushed to the `enrich:queue` stream.
4. **Enrichment** consumes `enrich:queue`, fans out to all five providers in parallel via `MultiProvider` (CrowdSec CTI, AbuseIPDB, GreyNoise, VirusTotal, Abuse.ch Feodo Tracker), merges results (worst reputation wins, highest confidence wins, categories union), and upserts `ip_enrichment`. Each provider's raw response is stored separately for forensics.
5. **Analytics** consumes the events stream:
 - `ScanDetector` flags an IP touching $\geq N$ distinct ports in a window → `scan_events`.

- AttackDetector flags brute force / post-auth commands / IDS exploit sigs → attack_events.
 - BehavioralProfiler maintains per-IP profiles + a threat score, clusters tactics.
 - Every 5 min, StatusAggregator writes a status_snapshots row and publishes it on Redis pub/sub status:live.
 - On schedule, ReportBuilder produces period reports.
6. **CrowdSec** independently parses the same logs, issues decisions; the firewall bouncer drops offenders at nftables. (Detection for analytics is decoupled from enforcement so a bad rule can't blind your dashboard.)
 7. **API** reads Postgres for REST, subscribes to status:live and pushes to the UI over WebSocket. **Frontend** renders; also polls /api/status every 5 min as a fallback.

7. Modularity / extension points (explicitly requested)

- **Intel providers — five built in, easily extended:** the system ships with five providers behind a common EnrichmentProvider interface:

Provider	Key required	Notes
crowdsec	Optional (CTI key)	Default single-provider mode; falls back to local LAPI decisions
abuseipdb	Yes	Free tier: 1 000 checks/day
greynoise	Optional	Community API works without a key, limited results
virustotal	Yes	Free tier: 500 lookups/day
abusech	No	Abuse.ch Feodo Tracker botnet C2 blocklist; no key needed
multi	—	Recommended — queries all providers in parallel and merges results

Merge logic: most severe reputation wins (malicious > suspicious > unknown > clean), highest confidence score wins, is_known_attacker is true if any provider flags the IP, geo/ASN uses the first non-null value, categories are the union of all providers. To add a custom provider: subclass EnrichmentProvider in enrichment/providers.py, decorate with @register, flip ENRICHMENT_PROVIDER=multi in .env.

- **Add a correlation/behavioral module:** subclass Detector in analytics/correlation/ (or a profiler in analytics/behavioral/), drop it in the registry. Engine auto-loads enabled detectors from config.
- **Add an AI module later:** the behavioral profiler exposes a score(profile, events) hook and the canonical raw transcripts are persisted, so an ML/LLM scorer can be added as its own container

that consumes the `events` stream and writes back to `attack_events.ai_score` — no schema migration needed.

- **Scale out:** Redis Streams + consumer groups means you can run N analytics workers, or lift the whole `data_net` tier to a second VM and point the pipeline's Redis at it. Postgres → managed PG; Cowrie → multiple sensors writing to the same bus.

8. Capacity plan (4 vCPU / 8 GB / 100 GB, 3-year retention)

Sized from **measured production rates** (23 days of live traffic, June–July 2026); the full disk model and its assumptions live in [docs/CAPACITY.md](#).

CPU / RAM budget

Compose limits are caps, not reservations; measured steady-state for the whole stack is **~2.5 GB RAM and <10% of one core**, so 4 vCPU / 8 GB carries a large burst margin (worst observed day: 171K events, ~50× steady state).

Service	CPU limit	RAM limit	Measured (steady state)
Suricata	1.5	2 GB	~50 MB (<code>detect.profile: low</code> , 2 AF_PACKET threads)
Cowrie + Extra	1.0	1 GB	~180 MB
CrowdSec (LAPI)	0.5	1 GB	~220 MB
Postgres/TimescaleDB	2	4 GB	~1.2 GB
Redis	0.25	1 GB	~15 MB (streams trimmed via MAXLEN)
Pipeline / Enrichment / Analytics	2	4 GB	~230 MB combined
API + Frontend + Caddy + docker-proxy	2	3.4 GB	~130 MB combined

A future AI module does not fit in the remaining RAM on this size — plan for a VM upgrade or remote inference when that lands. Add 2–4 GB swap as an OOM safety net.

Disk budget — 3 years of data retention

Postgres is the system of record. Raw sensor logs are rotated daily (`/etc/cron.daily/getarp-logs`, installed by `deploy/setup.sh`) and kept 14 days gzipped for forensics; without rotation `eve.json` alone would reach ~71 GB over 3 years.

Projection at 3× the measured steady-state rate (10K events/day, 300 new IPs/day):

Component	3-year size
events hypertable (columnar-compressed after 7 days, ~5×)	~3 GB
attack_events + scan_events	~1.5 GB
ips + ip_enrichment (~330K IPs, raw provider JSONB)	~3 GB
status_snapshots, profiles, reports, audit	~1.5 GB
WAL / bloat / vacuum headroom (×1.5)	→ ~13 GB DB total
Rotated sensor logs (14-day gzipped keep)	~2 GB
Docker images + build cache + capped container logs	~15 GB
OS + packages + journal	~10 GB
Working set → provision	~50 GB → 100 GB disk

Retention is enforced in three places: TimescaleDB retention policies (events, status_snapshots — 3 years, db/init.sql), the nightly retention_loop in analytics/engine.py for the plain tables (same horizon), and the log-rotation cron. Alert at 70% disk; pressure-relief levers (drop flow/dns from eve-log, trim enrichment raw JSONB) are listed in CAPACITY.md.

9. Security model (beyond the isolation principle)

- **Admin plane is not for the public.** The login/settings backend should be bound behind an allow-list or VPN. The Caddyfile ships with a commented @admin matcher + IP allow-list; turn it on. Public users get read-only dashboard routes only.
- **Move real SSH off 22.** The host's real sshd must listen on a high port bound to the management interface; port 22 belongs to Cowrie.
- **Least privilege containers:** honeypot containers run non-root where possible, read-only rootfs, cap_drop ALL, no Docker socket, resource limits to contain a fork-bomb attempt.
- **Secrets** via .env / Docker secrets, never in images. JWT secret + DB creds + provider API keys are all env-injected.
- **Retention & PII:** captured payloads can contain attacker (and occasionally third party) data. Retention is 3 years across the database (Timescale policies in db/init.sql + the analytics retention loop — see §8); passwords are redacted to length hints before storage. Document lawful basis since this is "Defence Intelligence."

10. Live status & reporting

- **Live (≤ 5 min):** StatusAggregator runs on a 300 s tick, snapshots active-attackers / new-IPs / events-per-min / top-ports / threat-level, publishes on `status:live`. UI updates over WebSocket instantly and via 5-min poll fallback.
- **Reports:** ReportBuilder renders period summaries (top attackers, scan vs. attack breakdown, new TTPs, geo/ASN distribution) to HTML/JSON, downloadable from the UI and generated on a daily schedule.

11. What is PoC-grade and would need hardening for production

Honest scope: this scaffold gives you a **working, modular system**. Before it's "production Defence Intelligence" you'd want: HA Postgres + backups, proper secrets manager, mTLS between services, signed/audited admin actions, SOAR/ticketing integration, formal data-retention & legal review, and load-testing Suricata against real line-rate traffic. Those are called out at each component's README.